Lecture 13 – Space complexity. Circuit complexity.

NTIN071 Automata and Grammars

Jakub Bulín (KTIML MFF UK)

Spring 2026

** Adapted from the Czech-lecture slides by Marta Vomlelová with gratitude. The translation, some modifications, and all errors are mine.*

Recap of Lecture 12

- time complexity, for TM as well as NTM
- the class P
- the class NP: verifier-based and nondeterminism-based definitions
- polynomial-time reductions
- NP-complete problems
- Cook-Levin theorem: SAT is NP-complete
- the class co-NP
- exponential time complexity

Space complexity

Space complexity

Similarly to time, measure space required for computation:

Definition

The space complexity of a [deterministic, single-tape] Turing machine that always halts is $f : \mathbb{N} \to \mathbb{N}$, where $f(n)$ is the maximum number of cells that M accesses on any input of size n.

For nondeterministic Turing machines we take the maximum over all computation paths.

But this does not work for sublinear space complexity, in particular, logspace (L) and nodeterministic logspace (NL).

The solution is to have two tapes: a read-only input tape, and a working tape whose space we measure. (If we want output, then a third 'write-only' output tape, only traverse in one direction.)

Classes of space complexity

For $f : \mathbb{N} \to \mathbb{R}^+$, define the space complexity classes:

$\mathrm{SPACE}(f(n)) = \{L \mid L \text{ decidable by a DTM in space } O(f(n))\}$
$\mathrm{NSPACE}(f(n)) = \{L \mid L \text{ decidable by an NTM in space } O(f(n))\}$

$\mathrm{L} = \mathrm{SPACE}(\log(n))$ note: $\log(n^k) \in O(\log(n))$
$\mathrm{NL} = \mathrm{NSPACE}(\log(n))$
$\mathrm{PSPACE} = \bigcup_k \mathrm{SPACE}(n^k)$
$\mathrm{NPSPACE} = \bigcup_k \mathrm{NSPACE}(n^k)$

Relationship between space and time

- Trivially, $\mathrm{TIME}(f(n)) \subseteq \mathrm{SPACE}(f(n))$ and $\mathrm{NTIME}(f(n)) \subseteq \mathrm{NSPACE}(f(n))$ (at most one cell per step)
- $\mathrm{NTIME}(f(n)) \subseteq \mathrm{SPACE}(f(n))$ (DFS on computation graph, store path from root as sequence of choices, not configs)
- If $f(n) \geq \log(n)$, then $\mathrm{NSPACE}(f(n)) \subseteq \mathrm{TIME}(2^{O(f(n))})$ (only $2^{O(f(n))}$ configs, fully construct computation graph + run BFS)

$$\mathrm{L} \subseteq \mathrm{NL} \subseteq \mathrm{P} \subseteq \mathrm{NP} \subseteq \mathrm{PSPACE} \subseteq \mathrm{NPSPACE} \subseteq \mathrm{EXPTIME}$$

Which inclusions are equalities? Which inclusions are strict?

- $\mathrm{PSPACE} = \mathrm{NPSPACE}$ (Savitch's theorem, coming up)
- $\mathrm{P} \neq \mathrm{EXPTIME}$ (Time hierarchy theorem, omitted)
- the rest is wide open

Example: NFA accepts everything?

$$\mathrm{ALL}_{\mathrm{NFA}} = \{\langle A \rangle \mid A \text{ is an NFA such that } L(A) = \Sigma^*\}$$

We can decide $\overline{\mathrm{ALL}_{\mathrm{NFA}}}$ in nondeterministic linear space:

- markers on states of A to keep track of set of possible states
- start with marker on start state
- repeat $2^{|Q|}$ times:
  - guess an input symbol $a \in \Sigma$
  - move markers according to transitions (simulate reading a)
  - if no marker on accepting state, accept
- if not accepted so far, reject
- note: $2^{|Q|}$ is enough, more means a loop that can be shortened

Thus $\mathrm{ALL}_{\mathrm{NFA}} \in$ co-NSPACE$(n) \subseteq$ co-SPACE$(n^2) =$ SPACE(n^2)

Is $\mathrm{ALL}_{\mathrm{NFA}}$ in NP $\cup$ co-NP?

Savitch's theorem

Savitch's theorem

Theorem (Savitch)

For any $f : \mathbb{N} \to \mathbb{R}^+$ *with* $f(n) \geq \log(n)$*:*

$$\mathrm{NSPACE}(f(n)) \subseteq \mathrm{SPACE}(f(n)^2)$$

Corollary

$\mathrm{PSPACE} = \mathrm{NPSPACE}$

Idea: Simulate the NTM deterministically by checking reachability in the configuration graph using a recursive algorithm that uses only $O(f(n)^2)$ space.

PATH in $\log^2(n)$ space

Given a directed graph G and $s, t \in V$, is there a path from s to t?

def $\textsc{path}(s, t)$:
  return $\textsc{reach}(s, t, |V|)$

$\textsc{reach}(s, t, k)$:
  if $k = 0$ **then return** $(s = t)$
  if $k = 1$ **then return** $(s = t)$ **or** $(s, t) \in E$
  for each $u \in V$ **do**
    if $\textsc{reach}(s, u, \lfloor k/2 \rfloor)$ **and** $\textsc{reach}(u, t, \lceil k/2 \rceil)$
      then return true
  return false

recursion depth $O(\log(n))$, each level $O(\log(n))$ bits to store k, u
total space $O(\log^2(n))$

Proof of Savitch's theorem

Given NTM M deciding L in space $g(n) \in O(f(n))$, and input w.

Assume M has a single final state and erases tape before accepting, so a single accepting configuration.

Is there a path from C_{init} to C_{accept} in the computation graph?

- $|V| \leq 2^{d \cdot g(n)}$ for some constant d
- $\log |V| \in O(g(n)) \subseteq O(f(n))$
- implement algorithm, start with $\text{REACH}(C_{\text{init}}, C_{\text{accept}}, 2^{d \cdot f(n)})$
- space $O(\log^2 |V|) \subseteq O(f(n)^2)$ $\square$

We were cheating

In the algorithm, we need to construct the value $f(n)$. How?

- Either assume the function $f(n)$ is space-constructible, i.e., there is a DTM computing $1^n \mapsto 1^{f(n)}$ in space $O(f(n))$.

  (Most common functions are space-constructible.)

- Or use a trick: Try the value $f(n) = 1$, if the simulated configurations run out of space, restart the whole process with $f(n) = 2$, and so on.

  Once we reach the correct value of $f(n)$, the algorithm will succeed; the space used does not change. □

PSPACE-complete problems

Quantified boolean formulas

A quantified boolean formula: $Q_1 x_1\, Q_2 x_2 \cdots Q_k x_k\, \varphi(x_1, \ldots, x_k)$ where $Q_i \in \{\forall, \exists\}$ and φ is a propositional formula.

Definition

$\mathrm{TQBF} = \{\langle \varphi \rangle \mid \varphi \text{ is a true QBF sentence}\}$

Note: equivalent to $\mathrm{QSAT} = \{\langle \varphi \rangle \mid \varphi \text{ is satisfiable QBF formula}\}$

Theorem

TQBF *is* PSPACE*-complete (under polynomial-time reductions).*

Proof: $\mathrm{TQBF} \in \mathrm{PSPACE}$ by space-efficient recursive evaluation

def $\textsc{eval}(\varphi)$:

if no variables **then** evaluate directly

if $\varphi = \exists x\, \psi$ **then return** $\textsc{eval}(\psi[x \leftarrow 0])$ **or** $\textsc{eval}(\psi[x \leftarrow 1])$

if $\varphi = \forall x\, \psi$ **then return** $\textsc{eval}(\psi[x \leftarrow 0])$ **and** $\textsc{eval}(\psi[x \leftarrow 1])$

Proof that TQBF is PSPACE-hard

Given $L \in \mathrm{PSPACE}$, decided by DTM M in space $f(n) \in O(n^k)$.
On input w, construct QBF sentence φ true iff M accepts w.

In particular, $\phi_{C_1,C_2,t}$ true iff there is a path of length $\leq t$ from C_1 to C_2 in the computation graph. Then set $\varphi = \phi_{C_{\text{init}},C_{\text{accept}},2^{d\cdot f(n)}}$.

For $k = 0$ easy, for $k = 1$ similar as in the proof of Cook-Levin.
For $k > 1$, naive approach: $\exists C_{\text{mid}}\ \phi_{C_1,C_{\text{mid}},\lfloor t/2 \rfloor} \wedge \phi_{C_{\text{mid}},C_2,\lceil t/2 \rceil}$ (where $\exists C_{\text{mid}}$ stands for "there exist vars describing a config" as in Cook-Levin), but this is too long, size $O(t) \subseteq O(2^{d\cdot f(n)})$.

Trick: universal quantifiers to reuse vars at each level of recursion

$$\exists C_{\text{mid}} \forall X, Y \in \{(C_1, C_{\text{mid}}), (C_{\text{mid}}, C_2)\}\ \phi_{X,Y,\lceil t/2 \rceil}$$

Length remains $O(f(n))$, nesting level $O(\log 2^{d\cdot f(n)}) = O(f(n))$, total length $O(f^2(n))$. □

$\forall X, Y \in \{(C_1, C_{\text{mid}}), (C_{\text{mid}}, C_2)\}$ is shorthand for $(X = C_1 \wedge Y = C_{\text{mid}}) \vee (X = C_{\text{mid}} \wedge Y = C_2) \rightarrow \ldots$

The Game problem

A game is $G = \langle V, E, A, B, s \rangle$ where $V = A \dot{\cup} B$ are positions of Players A and B, $E \subseteq V \times V$ are possible moves, and $s \in A$ is the starting position. Player loses if they have no move.

Theorem

$\mathrm{GAME} = \{\langle G \rangle \mid A \textit{ has winning strategy}\}$ *is PSPACE-complete.*

Proof: GAME is polynomial-time interreducible with TQBF

TQBF $\leq_p$ GAME: given φ, construct G where A chooses values for $\exists$ vars, B for $\forall$ vars; at the end A wins iff φ is true under the chosen assignment

GAME $\leq_p$ TQBF: given G, construct $\varphi = \psi_s$ that is true iff A has winning strategy from s. For each position v, define ψ_v:

$\psi_v = \bigvee_{(v,u) \in E} \psi_u$ if $v \in A$, and $\psi_v = \bigwedge_{(v,u) \in E} \psi_u$ if $v \in B$.

Note: $\bigvee_{\emptyset} = \bot$, $\bigwedge_{\emptyset} = \top$ defines base case for end positions. $\square$

Circuit complexity

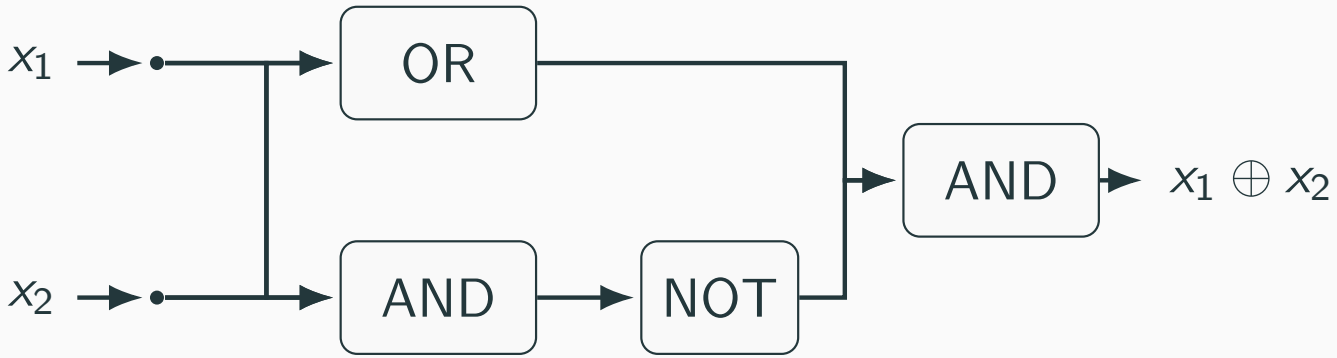
Boolean circuits

A Boolean circuit C is a directed acyclic graph whose nodes are

- input nodes (sources), labeled by input variables x_i
- gates, labeled by Boolean functions from some fixed basis, e.g. $\{\mathrm{AND}, \mathrm{OR}, \mathrm{NOT}\}$; sinks are called output gates

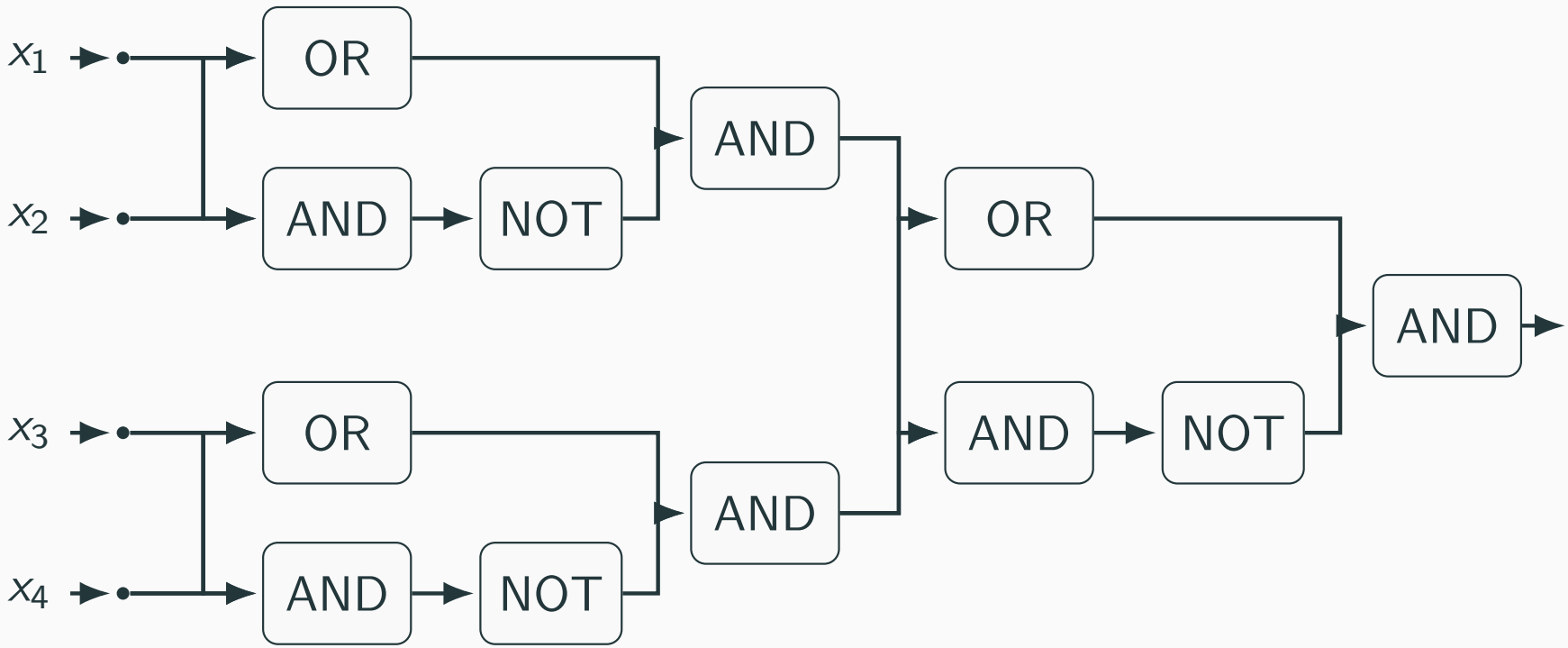
Each circuit C with n input nodes and m output gates computes a function $f_C : \{0,1\}^n \to \{0,1\}^m$; we focus on circuits with $m = 1$.

Example: parity on 2 bits $x_1 \oplus x_2 = (x_1 \vee x_2) \wedge \neg(x_1 \wedge x_2)$



size $= 4$ (#gates), depth $= 3$ (longest path from input to output)

Example: parity on 4 bits



Size for parity on n bits? $O(n)$

Depth? $O(\log n)$

Almost all functions require large circuits

Claim: Each $f : \{0,1\}^n \to \{0,1\}$ has a circuit of size $O(2^n)$.

Proof: Recursively express $f(x_1, \ldots, x_n)$ as

$$(x_1 \wedge f(1, x_2, \ldots, x_n)) \vee (\neg x_1 \wedge f(0, x_2, \ldots, x_n))$$

We get $s(n) = 2s(n-1) + 4$, $s(0) = O(1)$, so $s(n) \in O(2^n)$. □

Claim: Almost all functions on n bits require circuit size $\geq 2^n/4n$.

Proof: There are 2^{2^n} Boolean functions on n bits.

Each circuit with s gates can be described by a 01-string of length $s(2 + 2\log(s+n))$: for each gate specify label and two inputs. Hence for $s \geq n \geq 8$, there are $\leq 2^{3s \log s}$ circuits of size $\leq s$.

If $s < 2^n/4n$, then $2^{3s \log s} < 2^{\frac{3}{4}2^n}$ so only $2^{-2^{n-2}}$ fraction of functions can have circuit of size $\leq s$. □

Circuit complexity

A circuit family is a sequence $\{C_n\}_{n \geq 0}$ where C_n has n input nodes.

- size complexity of $\{C_n\}$ is the function $s : \mathbb{N} \to \mathbb{N}$ defined by $s(n) = \text{size}(C_n)$; similarly depth complexity of $\{C_n\}$

A circuit family $\{C_n\}$ decides a language $L \subseteq \{0,1\}^*$ if for every n and every $w \in \{0,1\}^n$, $w \in L \iff f_{C_n}(w) = 1$

- circuit complexity of L is the size complexity of the size-minimal circuit family deciding L
- circuit depth complexity of L is the depth complexity of the depth-minimal circuit family deciding L

Example: $PARITY = \{w \in \{0,1\}^* \mid |w|_1 \text{ is odd}\}$ has circuit complexity $O(n)$ and circuit depth complexity $O(\log n)$.

Complexity classes

$\mathrm{SIZE}(s(n)) = \{L \subseteq \{0,1\}^* \mid L \text{ has circuit complexity } O(s(n))\}$

- $\mathit{PARITY} \in \mathrm{SIZE}(n)$
- finite languages are in $\mathrm{SIZE}(1)$
- every language (even nonrecusive!), is in $\mathrm{SIZE}(2^n)$

$\mathrm{P/poly} = \bigcup_k \mathrm{SIZE}(n^k)$ i.e., decidable by polynomial circuit families

- Most languages are not in P/poly (counting argument)
- **Theorem:** $\mathrm{P} \subseteq \mathrm{P}/\mathit{poly}$ [next slide]
- Is $\mathrm{NP} \subseteq \mathrm{P}/\mathit{poly}$??
  - If not, $\mathrm{P} \neq \mathrm{NP}$
  - If yes, Polynomial Hierarchy collapses (as weird as P=NP)

Can you prove an explicit superpolynomial lower bound on circuit complexity of your favorite language from NP?

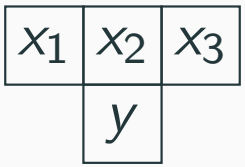
$\mathrm{P} \subseteq \mathrm{P}/poly$

Theorem

If $L \in \mathrm{TIME}(f(n))$ *where* $f(n) \geq n$*, then* $L \in \mathrm{SIZE}(f^2(n))$*.*

Proof overview: Consider the $f(n) \times f(n)$ table of configurations describing the accepting computation, as in proof of Cook-Levin.

Construct a circuit that checks its validity. For each cell plug in the same constant-size circuit verifying that the corresponding "tetris-shaped" frame is legal.



Enforce that 1st row is inital config, and 1st cell in last row is q_F (assume the TM returns before accepting). □

Corollary

If $L \in \mathrm{P}$*, then* $L \in \mathrm{P}/poly$*.*

Nonuniform vs. uniform computation

nonuniform = different algorithm for each input length

Warning: $L = \{1^n \mid n = \langle \mathrm{code}(M), w \rangle$ and M halts on input $w\}$ undecidable yet $\mathrm{SIZE}(1) \subseteq \mathrm{P}/poly$ (for each n, output correct bit)

$\mathrm{P}/poly$ = polynomial-time TM with advice $a\colon \mathbb{N} \to \{0,1\}^*$, on input w also receive string $a(n)$, $n = |w|$ (e.g. description of C_n)

uniform circuit families do not need advice:

Definition

A circuit family $\{C_n\}$ is uniform if there exists a deterministic Turing machine that, on input 1^n, outputs a description of C_n.

logspace-uniform = the Turing machine works in space $O(\log n)$

Theorem: P = logspace-uniform polynomial circuits

$\Rightarrow$ Build C_n from $\mathrm{P} \subseteq \mathrm{P}/poly$ proof on the fly in $O(\log n)$ space

$\Leftarrow$ TM that builds C_n + simulates its computation works in P $\square$

Parallel computation

A problem is efficiently parallelizable if can be solved by a computer with polynomially many processors in polylogarithmic time

- NC^i = languages decidable by logspace-uniform circuit family of polynomial size and depth $O(\log^i n)$
- AC^i same as NC^i but unbounded fan-in AND/OR gates
- $\mathrm{NC} = \bigcup_i \mathrm{NC}^i = \bigcup_i \mathrm{AC}^i$ (show $\mathrm{AC}^i \subseteq \mathrm{NC}^{i+1}$ by replacing unbounded fan-in gate with balanced tree of fan-in 2 gates)

Some facts:

- $\mathit{PARITY} \in \mathrm{NC}^1 \setminus \mathrm{AC}^0$ (rare explicit lower bound by Håstad)
- integer multiplication, matrix inverse are in NC (not obvious)
- $\mathrm{NC} \subseteq \mathrm{P}$ (simulate circuit in polynomial time)
- NC = efficiently parallelizable languages

Open: $P = NC$? (can all problems in P be efficiently parallelized?)

Summary of Lecture 13

- space complexity, L, NL, PSPACE, NPSPACE
- relationship between space and time
- Savitch's theorem, PSPACE $=$ NPSPACE
- PSPACE-complete problems: TQBF, Game
- circuit complexity